

Estruturas de Dados Básicas

Módulo 2: Fundamentos de Python para Manipulação de Dados

Funções básicas de E/S

E/S se refere a Entrada/Saída, ou seja, funções para receber dados do usuário via teclado ou arquivo (entrada) e mostrar dados a eles via tela/terminal ou arquivo (saída).

Saída

A função de saída mais comum é a `print()`, que exibe um conteúdo no terminal do usuário.

Podemos imprimir uma string diretamente:

```
print("Olá Mundo!")  
Olá Mundo!
```

Ou uma variável:

```
nome = "ABCD"  
print(nome)  
idade = 100  
print(idade)  
ABCD  
100
```

Trabalhando com `print()`

Ou uma string seguida de uma variável:

```
print("Meu nome é", nome)
print("5 x 5 =", 5*5)
Meu nome é ABCD
5 x 5 = 25
```

Podemos alternar entre strings e variáveis usando concatenação, mas temos que garantir que o valor da variável é transformado para string, senão recebemos um erro.

```
print("Meu nome é " + nome + " e tenho " + str(idade) + " anos")
# Receberíamos TypeError: can only concatenate str (not "int") to str
# se não colocássemos str() em idade

Meu nome é ABCD e tenho 100 anos
```

Para evitar ficar transformando variáveis em strings, podemos usar *f-strings*. Basta colocar um "f" antes da string e dentro dela colocar as variáveis dentro de chaves {}.

```
print(f"Meu nome é {nome} e tenho {idade} anos")
Meu nome é ABCD e tenho 100 anos
```

Entrada de Dados

A principal função é a `input()`, que pode conter uma string dentro (a mensagem que será mostrada logo antes de receber a entrada do usuário) e deve ser atribuída a uma variável.

```
entrada = input('Digite seu nome:')  
print(entrada)
```

Alec

Perceba que o código "espera" a entrada ser digitada e enviada para continuar sua execução.

Também é possível explicitar/converter o tipo da variável diretamente no input. Por padrão, a variável será uma string.

```
entrada = int(input('Digite sua idade:'))  
print(entrada)  
print(type(entrada))  
19  
<class 'int'>
```

Estruturas Básicas

Estruturas de dados são uma forma de armazenar e organizar dados, definindo a relação entre dados e as operações que podem ser executadas sobre eles.

Estruturas de dados podem ser divididas em três categorias:

- **Primitivas:** tipos de dados mais básicos, dos quais todas as outras estruturas de dados são compostas;
- **Não-primitivas:** tipos de dados mais complexos que contêm e manipulam tipos primitivos com operações específicas.

Estruturas primitivas já foram abordadas no módulo anterior. Vamos focar aqui nas não-primitivas.

Array

Array é uma *coleção de elementos de um mesmo tipo* em uma só variável, sendo pouco utilizado em python devido à existência de *listas* (que veremos a seguir).

Os elementos são armazenados **sequencialmente** (pense em uma fileira de caixas, uma após a outra) e acessados por um **índice** numérico, começando em 0; é comum dizer que arrays tem tamanho fixo.

Lista

Lista é uma coleção de itens que *podem ser de tipos diferentes*, também sendo armazenados sequencialmente e acessados por índices, mas tendo seu **tamanho variável**.

Em python, uma lista é identificada pelo uso do operador [] ou list().

```
minhaLista = list() # Uma lista vazia
minhaLista = []
print(minhaLista)
[]
```

Podemos inicializar uma lista com elementos de tipos diferentes, e acessar seus elementos indexando-a com uma posição.

```
minhaLista = [1, 'Arroz', True] # Lista inicializada com um inteiro, uma
string e um booleano
print(minhaLista)
print( minhaLista[0] ) # Acessando o primeiro elemento da lista
print( minhaLista[1] ) # Acessando o segundo elemento da lista
print( minhaLista[2] ) # Acessando o terceiro elemento da lista
[1, 'Arroz', True]
1
Arroz
True
```

Indexação em Listas

E se tentarmos acessar um elemento que não está na lista, passando um índice maior que seu tamanho?

```
# print(minhaLista[3]) # IndexError: list index out of range
```

Mas e se passarmos um índice negativo? Em python, um índice negativo simplesmente significa indexar a lista começando pelo seu fim

```
print( minhaLista[-1] ) # Último elemento da lista
print( minhaLista[-2] ) # Penúltimo elemento da lista
print( minhaLista[-3] ) # Antepenúltimo elemento da lista
True
Arroz
1
```

As principais operações envolvendo listas são mostradas a seguir. Note que usamos elas colocando um . após o nome da nossa lista, seguido do nome da operação e parênteses, dentro do qual passamos os argumentos dessa função. Não se preocupe, veremos funções mais para a frente.

Operações com Listas

.insert(índice, elemento): insere o elemento no índice indicado, "empurrando" os elementos à direita um índice para frente

```
# Nossa lista original é [1, 'Arroz', True]
minhaLista.insert(2, 'Uma string')
print(minhaLista)
[1, 'Arroz', 'Uma string', True]
```

.remove(elemento): remove o elemento da lista, se ele existir

```
minhaLista.remove('Arroz')
print(minhaLista)
[1, 'Uma string', True]
```

.pop(índice): remove o elemento no índice passado

```
minhaLista.pop(0)
print(minhaLista)
['Uma string', True]
```

list.reverse(lista): inverte a ordem dos elementos na lista passada. Note que aqui não estamos colocando um ponto após o nome da lista, mas sim passamos ela como parâmetro

```
list.reverse(minhaLista)
print(minhaLista)
[True, 'Uma string']
```


Mais Operações com Listas

len(lista): imprime a quantidade de elementos na lista

```
print(len(minhaLista))  
2
```

.clear(): apaga todos os elementos de uma lista

```
minhaLista.clear()  
print(minhaLista)  
[]
```

Tupla

Tuplas são similares a listas, com a diferença que elas **não podem ser modificadas**. Uma vez definida, um elemento não pode ser removido, alterado ou adicionado.

Tuplas são implementadas simplesmente ocultando o operador [] durante sua inicialização ou utilizando (), como mostrado a seguir; o modo de acesso a elementos é o mesmo que o de listas.

```
tupla_1 = 1,2,'Arroz',False  
print(tupla_1)  
tupla_2 = (1,2,'Arroz',False)  
print(tupla_2)  
print(tupla_2[2])  
(1, 2, 'Arroz', False)  
(1, 2, 'Arroz', False)  
Arroz
```

Note que tentar alterar um elemento da tupla gera erro.

```
# tupla_2[0] = 3 # TypeError: 'tuple' object does not support item assignment
```

Dicionário

É um conjunto de pares (chave, valor), onde cada chave é única (chaves repetidas não são permitidas) e identifica um valor.

Sua declaração é feita com {} ou dict().

```
meuDicionario = dict()
meuDicionario = {'Chave1':1, 'Chave2':2, 'Chave3':3}
print(meuDicionario)
{'Chave1': 1, 'Chave2': 2, 'Chave3': 3}
```

Um valor pode ser referenciado pela chave.

```
print( meuDicionario['Chave2'] )
2
```

Dicionários compartilham muitas funções com listas. Algumas funções específicas de dicionários são:

.items(): retorna uma lista com uma tupla para cada par

```
print( meuDicionario.items() )
dict_items([('Chave1', 1), ('Chave2', 2), ('Chave3', 3)])
```

Operações com Dicionários

.keys(): retorna uma lista com as chaves do dicionário

```
print( meuDicionario.keys() )  
dict_keys(['Chave1', 'Chave2', 'Chave3'])
```

.values(): retorna uma lista com os valores do dicionário

```
print( meuDicionario.values() )  
dict_values([1, 2, 3])
```

.update(): adiciona um par ao dicionário. Note que também podemos adicionar usando `dicionario[chave] = valor`

```
meuDicionario.update({'Chave4':4})  
print(meuDicionario)  
{'Chave1': 1, 'Chave2': 2, 'Chave3': 3, 'Chave4': 4}
```

.setdefault(): retorna o valor de uma chave específica; se ela não existir, insere a chave com o valor especificado

```
print( meuDicionario.setdefault('Chave4', 4) )  
print(meuDicionario)  
print( meuDicionario.setdefault('Chave5', 5) )  
print(meuDicionario)  
4  
{'Chave1': 1, 'Chave2': 2, 'Chave3': 3, 'Chave4': 4}  
5  
{'Chave1': 1, 'Chave2': 2, 'Chave3': 3, 'Chave4': 4, 'Chave5': 5}
```

Set

Listas que somente contém somente objetos/valores únicos. Valores não podem ser modificados uma vez inseridos, mas podem adicionados ou removidos. Os valores dentro de um set também não tem uma ordem específica e não podem ser acessados por um índice (é, de fato, um conjunto em sua definição matemática).

São declarados usando {} ou .set().

```
meuSet = set()
meuSet = {'A', 'B', 'C', 'D'}
print(meuSet) # Perceba que a ordem dos valores não se mantém
{'A', 'B', 'D', 'C'}
```

Sets possuem várias funções interessantes.

.add(valor): adiciona um elemento ao set

```
meuSet.add('E')
print(meuSet)
{'B', 'C', 'E', 'A', 'D'}
```

set1.difference(set2, set3, ...): retorna um set contendo os elementos que só existem no set1 e não no set2, set3 e etc.

```
meuOutroSet = {'A', 'B', 'C', 'X', 'Y', 'Z'}
print( meuOutroSet.difference(meuSet) )
{'X', 'Y', 'Z'}
```

Mais Operações com Sets

set1.difference_update(set2, set3, ...): remove de set1 os elementos que aparecem em set2, set3 e etc.

```
print(meuSet)
print(meuOutroSet)
meuSet2 = meuSet.copy()
meuSet2.difference_update(meuOutroSet)
print(meuSet2)
{'B', 'C', 'E', 'A', 'D'}
{'X', 'Z', 'Y', 'B', 'A', 'C'}
{'E', 'D'}
```

set1.intersection(set2, set3, ...): retorna um set contendo os elementos que só existem em todos os sets ao mesmo tempo

```
print(meuSet.intersection(meuOutroSet))
{'A', 'B', 'C'}
```

set1.intersection_update(set2, set3, ...): remove de set1 os elementos que não aparecem em set2, set3 e etc.

```
print(meuSet)
print(meuOutroSet)
meuSet3 = meuSet.copy()
meuSet3.intersection_update(meuOutroSet)
print(meuSet3)
{'B', 'C', 'E', 'A', 'D'}
{'X', 'Z', 'Y', 'B', 'A', 'C'}
{'A', 'B', 'C'}
```

Operações Avançadas com Sets

set1.isdisjoint(set2): retorna True se os sets não têm uma intersecção, e False se têm

```
print( meuSet.isdisjoint(meuOutroSet) )  
False
```

set1.issubset(set2): retorna True se todos os elementos de set1 estão em set2

```
print( meuSet3.issubset(meuSet) )  
True
```

set1.issuperset(set2): retorna True se todos os elementos de set2 estão em set1

```
print( meuSet.issuperset(meuSet3) )  
True
```

set1.union(set2): retorna a união dos dois sets

```
print( meuSet2.union(meuSet3) )  
{'E', 'B', 'A', 'C', 'D'}
```

Há ainda outros tipos de estruturas de dados mais avançados, como pilhas, deque, grafos e árvores.

Matrizes e Slicing

Matrizes é uma estrutura bidimensional formada por linhas e colunas (que nem na matemática mesmo). Em python, criamos matrizes aninhando vetores.

```
minhaMatriz = [ [1,4,5] , [-5,8,9] ]  
print(minhaMatriz)  
[[1, 4, 5], [-5, 8, 9]]
```

Por ser uma lista de listas, operamos com matrizes usando as funções e operadores de listas normais. A seguir estão alguns exemplos de acesso a elementos de uma matriz:

```
print(minhaMatriz[0])  
print()  
print(minhaMatriz[1])  
[1, 4, 5]  
[-5, 8, 9]  
  
print(minhaMatriz[1][0]) # Pense nisso como (minhaMatriz[1])[0] => ([-5, 8, 9])[0] => -5  
print(minhaMatriz[1][1])  
print(minhaMatriz[1][2])  
-5  
8  
9
```

Apenas pense em cada linha da matriz como uma lista, e você conseguirá operar com matrizes sem problemas. E não se preocupe; elas ficarão mais fáceis com algumas ferramentas que veremos mais pra frente.